ESTUDO TÉCNICO · PU-EST-001

Teste de carga para uma rede de 800 mil usuários

Como converter usuários cadastrados em carga simultânea, o que medir, com quais critérios de aprovação e em que ordem executar

Documento	PU-EST-001, versão 0.1 (rascunho para decisão)
Data	23 de setembro de 2026
Autor	Lucas Salomão
Relaciona-se a	PU-DOC-001: **RNF-01**, riscos `R-01` e `R-17`, Capítulo 17 (SLOs)
Estado	Nenhum teste executado. Os números são estimativas com premissas declaradas, feitas para serem substituídas por medição.

Sumário

1 Resumo executivo **1**

2 Por que 800 mil não é o número do teste **3**

2.1 Cadastrado, ativo, simultâneo . . . 3

2.2 A Lei de Little . . . 3

2.3 Modelo aberto, não fechado . . . 4

3 Da rede ao número: o modelo de carga **5**

3.1 Premissas . . . 5

3.2 Simultâneos no horário mais cheio . . . 6

3.3 A parede das 8h: taxa de *login* . . . 6

3.4 Requisições por segundo em regime . . . 7

4 O que testar **9**

4.1 As jornadas . . . 9

4.2 Fora do escopo desta rodada . . . 10

5 Hipóteses de gargalo que o código já aponta **11**

6 Os tipos de teste e a ordem de execução **13**

6.1 Capacidade unitária: o teste que mais ensina . . . 13

6.2 Carga híbrida: protocolo mais navegador . . . 14

7 Critérios de aprovação e de parada **15**

7.1 Aprovação: os SLOs que a plataforma já declarou . . . 15

7.2 Parada: quando interromper . . . 16

7.3 O que observar do lado do servidor . . . 16

8 Ambiente, dados e ferramentas 17

8.1 Onde rodar . . . 17

8.2 Dados sintéticos, nunca reais . . . 18

8.3 IA: simulada em escala, real em amostra . . . 18

8.4 Banda da escola . . . 19

8.5 Ferramenta . . . 19

9 Contas de capacidade para levar ao teste 21

9.1 Conexões com o banco . . . 21

9.2 IA pela Lei de Little . . . 21

9.3 *Login* . . . 22

10 Decisões que o teste não toma 23

11 Plano de execução 24

A Esqueleto do roteiro k6 25

B Referências de método sem citação direta no texto 27

1

Resumo executivo

A pergunta: a rede informou 800 mil usuários. Como testar a carga de uma plataforma quando é óbvio que esses 800 mil não entram ao mesmo tempo?

A resposta curta: o número de cadastrados não é a carga. A carga é a *taxa de chegada* de sessões e de requisições, e ela se deriva de três coisas: quantos estudantes estão na escola no turno mais cheio, quantas turmas usam uma missão na mesma aula, e quanto tempo dura uma sessão. A Lei de Little ($N = \lambda W$) liga essas grandezas (LITTLE, 1961), e a literatura de teste de desempenho manda modelar *sessões* em vez de "usuários simultâneos" (MEIER et al., 2007a), com pelo menos três cenários de uso (esperado, melhor e pior caso).

> Com as premissas do Capítulo 3, a rede de 800 mil gera, no horário mais cheio:
> - **cenário esperado**: ~50.000 estudantes simultâneos em missão, ~210 *logins* por segundo no primeiro minuto da aula e ~40.000 requisições por segundo à API em regime;
> - **cenário baixo**: metade disso; **cenário alto**: o dobro;
> - e um **teste de ruptura** que continua subindo até quebrar, para saber quanta folga existe além do cenário alto.

Três conclusões mudam o plano, e aparecem antes de qualquer teste rodar:

1. **A premissa de arquitetura foi escrita para outra escala.** A PU-DOC-001 dimensiona ~10.000 estudantes e 2.000–4.000 simultâneos *por contrato*, numa "instância pequena" de Cloud SQL. Uma rede de 800 mil numa única *stack* é 80 vezes isso. Antes do teste, é preciso decidir se a rede é um contrato (uma *stack*) ou vários (Capítulo 10).
2. **O código já aponta os gargalos prováveis**, e o teste deve começar por eles: o *hash* de senha roda em JavaScript no *event loop* (~100 ms por *login*); o *polling* de 5 s por estudante passa pela guarda de liberação, que faz de 5 a 6 *queries*; o painel do professor relê todos os eventos da atividade a cada 5 s; e o *pool* de conexões do Prisma está no padrão (Capítulo 5).
3. **O gargalo do estudante pode nem estar na nuvem.** A média de banda nas escolas públicas monitoradas é de 0,26 Mbps por aluno, e só 11% delas atingem a meta de 1 Mbps por aluno (Comitê Gestor da Internet no Brasil, 2024). O teste de navegador precisa simular a rede da escola, não a fibra do escritório.

O que se propõe: k6 como ferramenta, carga em *modelo aberto* (taxa de chegada, não número fixo de usuários virtuais), sete testes em sequência (*smoke*, capacidade unitária, carga média, a “parede das 8h”, estresse e ruptura, resistência e IA isolada), mais uma pequena fração de navegadores reais medindo Web Vitals. Os critérios de aprovação são os SLOs que a plataforma já declarou no Capítulo 17 da PU-DOC-001.

2

Por que 800 mil não é o número do teste

2.1 Cadastrado, ativo, simultâneo

São três populações diferentes, e confundir as duas primeiras com a terceira é o erro mais comum ao planejar um teste de carga:

População	Definição	Ordem de grandeza
Cadastrados	Contas existentes no banco. Determinam o *volume de dados* (tamanho de tabela, índice, *backup*), não a carga.	800.000
Ativos no dia	Quem entra pelo menos uma vez num dia letivo. Depende da adoção pedagógica: quantas aulas por semana usam uma missão.	dezenas a centenas de milhares
Simultâneos	Quem tem uma sessão aberta *no mesmo instante*. É o que ocupa instância, conexão e memória.	dezenas de milhares

O guia de testes de desempenho da Microsoft é direto sobre isso: em vez de modelar a experiência com base em usuários simultâneos, "*it is more useful to base your model on user sessions*", e o exemplo do capítulo mostra 23 sessões numa hora produzindo de 9 a 10 usuários simultâneos (MEIER et al., 2007a). O *syllabus* de desempenho do ISTQB faz o mesmo alerta com um exemplo numérico: 500 usuários que fazem uma consulta curta por minuto geram 30.000 consultas por hora; os mesmos 500 fazendo uma por hora geram 500 — uma diferença de 60 vezes com o "mesmo número de usuários" (International Software Testing Qualifications Board, 2018).

2.2 A Lei de Little

A ponte entre sessões e simultaneidade é a Lei de Little (LITTLE, 1961):

$$N = \lambda \cdot W \tag{2.1}$$

em que N é o número médio de sessões abertas, λ é a taxa de chegada de sessões (sessões por segundo) e W é a duração média de uma sessão. Ela vale para qualquer sistema estável, sem hipótese sobre a distribuição das chegadas. O ISTQB traz a mesma relação escrita para usuários virtuais: vazão $=$ usuários $/$ (tempo de processamento $+$ tempo de pensamento) (International Software Testing Qualifications Board, 2018).

No contexto da plataforma, isso se lê assim: se as turmas abrem missões de 45 minutos, uma taxa de chegada de 18 estudantes por segundo sustenta cerca de 50.000 sessões simultâneas ($18 \times 2700\,\text{s} \approx 48.600$). E o inverso: se a concentração do início da aula faz λ disparar por alguns minutos, é esse disparo — não a média — que derruba o *login*.

2.3 Modelo aberto, não fechado

Há duas formas de gerar carga (SCHROEDER; WIERMAN; HARCHOL-BALTER, 2006):

- **modelo fechado**: um número fixo de usuários virtuais, cada um só manda a próxima requisição quando a anterior volta;
- **modelo aberto**: as chegadas acontecem numa taxa definida, *independentemente* de o servidor ter respondido.

Schroeder, Wierman e Harchol-Balter mostram que, para a mesma carga, o tempo de resposta médio é *significativamente menor* no modelo fechado do que no aberto (SCHROEDER; WIERMAN; HARCHOL-BALTER, 2006). O motivo é que, no modelo fechado, um servidor lento *desacelera o próprio gerador*: os usuários virtuais esperam, a taxa de chegada cai, e a latência medida sai otimista. Gil Tene chamou esse efeito de *coordinated omission*: a maior parte dos artefatos de alta latência do servidor simplesmente não é registrada (TENE, 2013; TENE, 2026). A documentação do k6 descreve o mesmo sintoma e recomenda os executores de taxa de chegada (`constant-arrival-rate` e `ramping-arrival-rate`) para o modelo aberto (Grafana Labs, 2026e).

> **DECISÃO DE PROJETO**
>
> **O teste usa o modelo aberto.** As crianças não esperam a API responder para chegar à escola: às 7h30 elas entram na sala e o professor pede que abram a missão, com o servidor lento ou não. Isso é um sistema aberto por natureza. O modelo fechado entra só onde ele descreve o comportamento real — o *polling* de uma aba já aberta, que espera a resposta anterior.

3

Da rede ao número: o modelo de carga

3.1 Premissas

Toda estimativa abaixo é uma cadeia de premissas. Elas estão numeradas para que a rede possa corrigir cada uma isoladamente, e o resultado se recalcula sem reescrever o método.

ID	Premissa	Valor	Origem
P1	Usuários cadastrados	800.000	Informado pela rede.
P2	Fração de estudantes	95% (760.000)	Premissa; o resto são professores e gestores. **Pedir o número à rede.**
P3	Estudantes no turno mais cheio	55% (~420.000)	Premissa. O Censo Escolar 2024 mostra que 99,9% dos anos finais do fundamental e 82,5% do ensino médio estudam no diurno, e que 22,9% das matrículas públicas são de tempo integral (Instituto Nacional de Estudos e Pesquisas Educacionais Anísio Teixeira, 2025); a divisão manhã/tarde não é publicada em agregado e tem de vir dos microdados ou da rede.
P4	Tamanho médio de turma	30	Premissa, coerente com o `R-17` da PU-DOC-001.
P5	Aulas por turno e por semana	5 por dia, 25 por semana	Premissa de grade típica.
P6	Aulas por semana, por turma, que usam uma missão (k)	1, 2 ou 4	**É a premissa que mais pesa**, e é pedagógica: depende de quantas missões o currículo da rede encaixa por semana.
P7	Fator de concentração (f)	1,5	Premissa: a aula mais disputada tem 50% mais turmas usando a plataforma que a média (primeira aula, por exemplo).
P8	Duração média de uma sessão (W)	45 min	Uma hora-aula de 50 minutos, menos a acomodação da turma.

ID	Premissa	Valor	Origem
P9	Janela de entrada	50% nos 2 primeiros minutos, o resto em 8	Premissa do “podem abrir agora” do professor. É a “parede” descrita na PU-DOC-001.

3.2 Simultâneos no horário mais cheio

A fração de turmas numa aula com missão é $k/25$, e a simultaneidade no horário mais cheio é:

$$N_{\text{sim}} = E_{\text{turno}} \times \frac{k}{25} \times f \tag{3.1}$$

Cenário	k	Simultâneos	Turmas ativas	Leitura
A — baixo	1	25.200	840	Início do contrato, adoção parcial.
B — esperado	2	50.400	1.680	Duas missões por turma por semana.
C — alto	4	100.800	3.360	Adoção plena, ou um “dia D” pedido pela secretaria.
D — referência documental	—	152.000	5.070	A razão da PU-DOC-001 (20% dos cadastrados, o piso de 2.000–4.000 em 10.000) aplicada a 760.000.

> **NOTA**
>
> **Por que o cenário D não é o alvo, mas entra no teste.** A razão de 20–40% de simultâneos sobre cadastrados foi escrita para uma rede pequena e não é plausível em 800 mil: o TIC Educação e o NIC.br mostram que só 29% das escolas públicas têm dispositivo para os alunos, cerca de um para cada dez, e que o uso do celular pessoal na escola caiu de 55% para 45% (Comitê Gestor da Internet no Brasil, 2024; Centro Regional de Estudos para o Desenvolvimento da Sociedade da Informação, 2025). Os dispositivos são um teto natural da simultaneidade. Mesmo assim, o cenário D serve de marco no teste de ruptura: saber se a plataforma chega nele, ou onde para antes, custa só mais alguns minutos de rampa.

3.3 A parede das 8h: taxa de *login*

Pela premissa P9, metade das sessões da aula chega nos dois primeiros minutos:

Cenário	Pico (2 min)	Restante (8 min)	CPU só de *hash*
A	105 *logins*/s	26/s	~10,5 vCPU no pico
B	210 *logins*/s	52/s	~21 vCPU no pico
C	420 *logins*/s	105/s	~42 vCPU no pico

A última coluna usa o custo medido em desenvolvimento: ~100 ms de CPU por comparação `bcryptjs` com custo 10 (`apps/hub/lib/auth.ts:41-43`). É o pior caso — quem já tem sessão válida de 2 h não refaz *login* —, e é por isso que ele é o pior caso certo para testar: o primeiro turno do dia começa com todas as sessões vencidas.

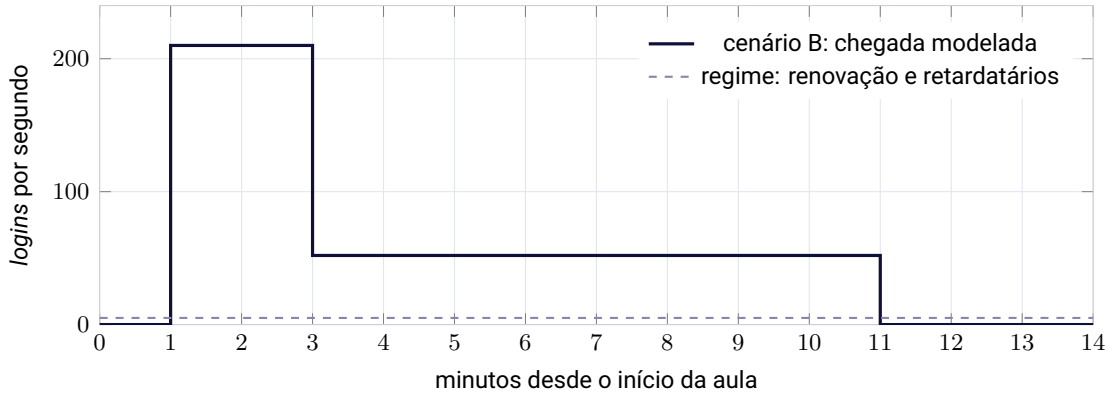


Figura 3.1 – O perfil de chegada que o teste da parede reproduz: um degrau de dois minutos, a cauda de oito, e o regime depois. O Google recomenda testar tanto a rampa quanto o degrau, porque o efeito de *cache* faz os dois darem resultados diferentes (ULRICH, 2016).

3.4 Requisições por segundo em regime

Depois que a missão abre, cada aba de estudante gera tráfego próprio. O levantamento do código (Capítulo 5) dá a composição:

Fonte	Taxa por aba	Onde
Polling de estado (fase, relógio, portão)	1 a cada 5 s	`exp08ef1b`, `exp02em1b`, `exp06ef2b`
Eventos do motor (Instanooria)	lote a cada 4 s	`exp08ef1b/src/lib/eventos.ts:51-52`
Telemetria	até 6 por minuto	`packages/sdk/src/telemetry.ts`
Ações da missão	~1 por minuto	rotas `POST` de cada etapa
Preflight CORS	até $\times 2$	`apps/api/src/main.ts:48`: sem `maxAge`, o navegador guarda o *preflight* por poucos segundos

Somado, dá de 0,35 a 1,2 requisições por segundo por aba; o valor central de 0,8 dá ~20.000 req/s no cenário A, ~40.000 no B e ~80.000 no C. Esse é o número que dimensiona a API, e ele é dominado pelo *polling*, não pelo que a criança faz. A documentação do k6 recomenda justamente descrever carga de API por taxa de requisições, e converter para taxa de iterações dividindo pelo número de requisições por iteração (Grafana Labs, 2026a).

> **ATENÇÃO**
>
> Estes números são ordem de grandeza. O objetivo deles é escolher a escala do gerador e da cota de nuvem, não prever o resultado. O primeiro teste (capacidade unitária, Seção 6.1) substitui a estimativa pela medida.

4

O que testar

4.1 As jornadas

O teste não exercita rotas soltas: exercita as jornadas que acontecem numa aula, na proporção em que acontecem. Uma jornada é uma sequência de requisições com tempo de pensamento entre elas (MEIER et al., 2007a).

J	Jornada	Sequência (requisições reais)	Peso
J1	*Login* do estudante	`GET /login` → `/api/auth/providers` → `/api/auth/csrf` → `POST /api/auth/callback/credentials` (que chama `POST /auth/login` na API) → `/api/auth/session` → `GET /estudante` (que chama `GET /salas/minhas`). Cerca de 8 requisições ao hub e 2 à API.	1 por sessão
J2	Abrir a missão	`GET /exp/:expId`: `/salas/minhas`, `/experiencias/:expId`, `/recursos/efetivos` (3 chamadas à API, 2 assinaturas RS256) → *bundle* da CDN → *handshake* `exp:ready`/`hub:init` → rota de entrada da experiência (passa por `exigirMissaoLiberada`).	1 por sessão
J3	Missão sem IA (Rastro do Boato, `exp06ef2b`)	`GET /rastro-boato/sessao`, `POST /entrar`, *polling* de `/gate` a cada 5 s, `POST` de auditoria, decisão, modelagem e reflexão.	50%
J4	Missão com motor de eventos (Instanooria, `exp08ef1b`)	`GET /runs/boot`, `POST /runs`, `/entrar`, `/feed`, *polling* de `/fase` a cada 5 s, `POST /runs/:id/eventos` a cada 4 s, `student-post` (IA opcional).	35%
J5	Missão com IA síncrona (unooGPT, `exp02em1b`)	`GET /unoogpt/sessao`, *polling* de `/relogio`, `POST /unoogpt/pergunta` (10–30 s, até 6 voltas de ferramenta).	15%
J6	Painel do professor	Uma aba por turma ativa: monitor a cada 5 s (`GET /atividades/:id/monitor`) ou `router.refresh()` a cada 5–10 s, conforme a missão.	1 por turma

J	Jornada	Sequência (requisições reais)	Peso
J7	Renovação do *token*	`GET /api/exp-token` a cada 10 minutos por aba aberta.	contínuo
J8	Telemetria	`POST /telemetry/batch` → Pub/Sub → `POST /telemetry/ingest`. Mede-se a fila, não só a rota.	contínuo

Os pesos de J3 a J5 são premissa e devem refletir o calendário pedagógico da rede no mês do teste. O que importa é que as três classes de missão estejam presentes: a que só lê e grava, a que grava eventos em alta frequência e a que segura a requisição por dezenas de segundos esperando a IA.

4.2 Fora do escopo desta rodada

- **O provedor de identidade (WSO2).** Hoje o hub é o emissor, e o WSO2 entra depois. Quando entrar, precisa de teste próprio: em agosto de 2020, o primeiro dia de aulas de distritos de San Antonio (EUA) caiu às 6h45 porque o portal de *login* não aguentou o volume, e a falha arrastou em cascata o Google Classroom e o Zoom, que dependiam dele (CARNETT, 2020).
- **A CDN em si.** Ela é um serviço gerenciado com capacidade muito acima da nossa; o que interessa medir é a taxa de acerto de *cache*, porque o Google avisa que nenhuma requisição tem garantia de sair do *cache* (Google Cloud, 2026b). Seguindo a recomendação do k6, os testes de protocolo excluem os *assets* da CDN e os testes de navegador os incluem (Grafana Labs, 2026d).
- **Telas de gestão** (importação, relatórios da diretoria): baixo volume, com limite próprio; entram só no teste de resistência como ruído de fundo.

5

Hipóteses de gargalo que o código já aponta

Um teste de carga bem feito começa com hipóteses: sem elas, a equipe olha o painel no dia e não sabe o que procurar. O levantamento do código em 23 de setembro de 2026 aponta oito, e cada teste do Capítulo 6 existe para confirmar ou descartar pelo menos uma delas. **Nenhuma foi medida ainda**: são leituras do código.

H	Hipótese	Por que o código sugere	Teste que decide
H1	O *login* satura a CPU da API	`bcryptjs` (JavaScript puro) com custo 10, no *event loop*: ~100 ms por comparação, e uma instância Node faz no máximo ~10 por segundo enquanto bloqueia todo o resto (`apps/api/src/auth/auth.controller.ts:141,154-167`).	Parede das 8h
H2	A busca do *login* não tem índice	`user.findMany({username, ativo})`, mas o único índice é o `UNIQUE (escola_id, username)` (`schema.prisma:442`). Com 800 mil linhas, a busca sem `escola_id` pode varrer a tabela.	Capacidade unitária
H3	O *pool* de conexões esgota antes da CPU	Nenhum `connection_limit` no código: vale o padrão do Prisma ($2 \times$ vCPU $+ 1$). Cada `comTenancy` abre uma transação interativa (há 239 chamadas), com `maxWait` de 2 s. O *pooler* está declarado na documentação e não aparece no repositório.	Carga média, estresse
H4	O *polling* multiplica a guarda de liberação	Toda rota de estudante, inclusive o *polling* de 5 s do portão, passa por `exigirMissaoLiberada`, que faz de 5 a 6 *queries* (`apps/api/src/comum/liberacao.ts:104-130`). No cenário B, se todos estivessem nessa missão, seriam ~60.000 *queries*/s só de guarda.	Carga média

H	Hipótese	Por que o código sugere	Teste que decide
H5	O monitor do professor cresce durante a aula	`GET /atividades/:id/monitor` a cada 5 s lê todas as *runs* e todos os eventos da atividade, sem `take` (`apps/api/src/analise/metricas.ts:123-160`). O custo por chamada aumenta a cada minuto de aula.	Resistência
H6	A IA prende a capacidade da API	`POST /unoogpt/pergunta` leva 10–30 s. Pela Lei de Little, 5 mil estudantes perguntando uma vez a cada 5 minutos, com 20 s de espera, mantêm ~330 requisições em voo; a largada simultânea da turma multiplica isso por 5. Cada uma ocupa um dos 80 lugares de concorrência da instância.	IA isolada
H7	Limites em memória viram N vezes o declarado	O *rate limit* e as quotas locais são um `Map` por instância (`apps/api/src/comum/rate-limit.ts`). Com 50 instâncias, o “8 por minuto” vira até 400. Não é gargalo: é proteção que some sob carga.	Estresse
H8	O hub assina demais	Cada chamada do hub à API assina um JWT RS256 novo (`apps/hub/lib/api-servidor.ts:80,121`), e cada abertura de missão faz duas. A API verifica cada uma.	Parede das 8h

> **NOTA**
>
> O teste não corrige nada disso: ele mede. A regra do projeto é “nenhuma peça de infraestrutura sem gargalo medido” (RE-01 da PU-DOC-001), e esta lista existe para que o gargalo, quando aparecer, já tenha um nome e um arquivo.

6

Os tipos de teste e a ordem de execução

A classificação segue a documentação do k6 (Grafana Labs, 2026c) e o *syllabus* do ISTQB (International Software Testing Qualifications Board, 2018). A ordem importa: cada teste só roda se o anterior passou, porque um teste de estresse num sistema que falha no *smoke* só mede o defeito já conhecido.

#	Teste	Carga e duração	Pergunta que responde
1	*Smoke*	5–10 usuários, 2 min	O roteiro funciona? *Login*, *token*, missão e painel respondem 2xx?
2	Capacidade unitária	1 instância fixa da API e do hub, rampa até quebrar	Quantas req/s *uma* instância aguenta dentro do SLO? É o número que dimensiona tudo.
3	Carga média	Cenário B, 30–60 min	O sistema se mantém dentro do SLO na carga esperada?
4	Parede das 8h (*spike*)	Degrau da Figura 3.1, 15 min	O *autoscaling* acompanha o degrau? Há 429 ou 5xx no primeiro minuto?
5	Estresse e ruptura (*breakpoint*)	Cenário C, depois rampa até D e além	Onde quebra, e como? Degrada com 429 e volta sozinho, ou cai em cascata?
6	Resistência (*soak*)	Cenário A ou B, 5 h (um turno inteiro)	Vazamento de memória, *pool* que não devolve conexão, fila de telemetria que cresce, monitor do professor ficando lento.
7	IA isolada	Turmas inteiras de J5 largando juntas	`R-17`: quantos turnos simultâneos cabem na cota do Vertex e na concorrência da API?

6.1 Capacidade unitária: o teste que mais ensina

Com o *autoscaling* desligado (mínimo e máximo de instâncias iguais a 1), sobe-se a taxa de chegada até que o p95 passe do SLO ou a taxa de erro passe de 1%. O resultado é a capacidade

c de uma instância, em req/s. A partir dela:

$$\text{instâncias necessárias} = \frac{\text{req/s do cenário}}{c \times 0{,}6} \tag{6.1}$$

em que 0,6 vem do alvo de 60% de CPU e de concorrência que o *autoscaler* do Cloud Run persegue (Google Cloud, 2026a). O Google recomenda repetir o teste em cargas diferentes para confirmar que o serviço escala de forma linear (Google Cloud, 2026f) — e ele não escala: a Lei Universal de Escalabilidade de Gunther descreve por que contenção e coerência fazem a curva dobrar (GUNTHER, 2007). É por isso que o teste 3 existe mesmo depois do 2.

6.2 Carga híbrida: protocolo mais navegador

O k6 recomenda gerar a maior parte da carga em nível de protocolo e rodar, junto, um número pequeno de navegadores reais (Grafana Labs, 2026d). Os navegadores medem o que o protocolo não vê — o LCP, o INP e o tempo até o *handshake* da ponte — e o módulo de navegador do k6 exporta essas métricas diretamente (Grafana Labs, 2026b). Proposta: 50 a 100 navegadores durante os testes 3 e 4, com a rede limitada a um perfil de escola (ver Seção 8.4).

7

Critérios de aprovação e de parada

7.1 Aprovação: os SLOs que a plataforma já declarou

O teste não inventa critério: usa os SLOs do Capítulo 17 da PU-DOC-001, porque são as promessas que a operação vai cobrar depois. O livro de SRE do Google insiste em percentis, não em médias, porque a média esconde a cauda que o usuário sente (JONES et al., 2016). No k6, cada linha vira um *threshold*, que é o critério de aprovação ou reprovação do teste (Grafana Labs, 2026g).

Critério	Limite	Origem
Erros 5xx na API e no hub	$< 0{,}5\%$	SLO de disponibilidade de 99,5%
Latência de `/runs/:id/feed`, `/experiencias/:id`, `/auth/*`	p95 < 800 ms	SLO de latência da API
Login completo (J1 inteira)	p95 < 3 s	Proposta deste estudo
Handshake da ponte	99% em até 8 s	SLO da ponte
Turno de IA	98% concluídos; p95 < 30 s	SLO de IA e alerta B
Fila de telemetria	mensagem mais antiga < 5 min	SLO de ingestão
LCP e INP (navegador)	p75: LCP $\leq 2{,}5$ s, INP ≤ 200 ms	Core Web Vitals (Google, 2026)
Respostas 429 no degrau	$< 1\%$ no primeiro minuto	O Cloud Run segura requisições por até 10 s sem instância livre e depois responde 429 (Google Cloud, 2026a)

7.2 Parada: quando interromper

- Taxa de erro acima de 5% por mais de um minuto: o k6 aborta sozinho com `abortOnFail` (Grafana Labs, 2026g).
- CPU do Cloud SQL acima de 90% por cinco minutos, ou conexões no limite: o banco é compartilhado por todas as escolas e não se recupera sozinho de uma fila de conexões.
- **O gerador passou de 80% de CPU ou 90% de memória.** Aí o resultado é do gerador, não do sistema (Grafana Labs, 2026f).
- Gasto de IA acima do teto aprovado para o teste.

7.3 O que observar do lado do servidor

A observabilidade do Capítulo 17 já está no ar e é o que torna o teste legível. Durante cada rodada:

- **Cloud Run**: instâncias, concorrência por instância, CPU, memória, latência de *startup*, contagem de 429;
- **Cloud SQL**: CPU, conexões ativas, esperas por *lock*, `pg_stat_statements` (as dez *queries* mais caras), Query Insights;
- **API**: erros `P2024` do Prisma (espera por conexão do *pool*), *queries* acima de 500 ms (o log lento já existe), latência de IA por rota;
- **Pub/Sub**: `oldest_unacked_message_age` e fila morta;
- **Vertex**: respostas 429 por cota.

O Google recomenda exportar os logs para o BigQuery e analisar a latência por lá depois do teste (Google Cloud, 2026f).

8

Ambiente, dados e ferramentas

8.1 Onde rodar

> **DECISÃO DE PROJETO**
>
> **Na *stack* de produção da rede, antes do lançamento, com dados sintéticos, e apagando tudo depois.** Até novembro de 2026 a produção não tem nenhum estudante real: é a única janela em que dá para testar a configuração exata que vai atender a rede (*tier* do banco, cotas, rede, CDN) sem arriscar dado de menor. Depois do lançamento, o teste passa para uma cópia da *stack* em homologação, e nunca volta para produção.

- **Região**: a de produção (`southamerica-east1`). O Google sugere `europe-west1` para testes de carga no Cloud Run por causa da cota (Google Cloud, 2026f), mas testar noutra região mede outro sistema; a alternativa é pedir aumento de cota na região real.
- **Cotas**: o máximo padrão do Cloud Run é de 100 instâncias por serviço, e o Google pede que se fale com a equipe da conta quando o teste for passar disso (Google Cloud, 2026f). É provável que o cenário B já passe.
- **Aviso ao Google**: nenhuma fonte oficial exige aviso prévio para teste de carga nos próprios projetos; o FAQ de segurança dispensa até o de teste de invasão, desde que respeitada a política de uso aceitável (Google Cloud, 2026c). O que existe é a conversa de cota acima.
- **Gerador**: não usar o Pub/Sub para gerar carga, que o próprio Google desaconselha (Google Cloud, 2026f). Usar máquinas na mesma região, fora da VPC da aplicação, para que o tráfego entre pelo *load balancer* como o de uma escola.

8.2 Dados sintéticos, nunca reais

> **REGRA INVIOLÁVEL**
>
> Nenhum dado real de estudante entra no teste, nem anonimizado. A LGPD exige que o tratamento de dados de crianças e adolescentes seja feito no melhor interesse deles (art. 14) e limitado ao mínimo necessário (art. 6º, III), e o dado anonimizado volta a ser pessoal se a anonimização for revertida (art. 12) (Brasil, 2018). Um teste de carga não precisa de nenhuma pessoa real.

A massa é gerada pelo mesmo caminho da produção — o importador (ADR-54 e ADR-59), que é quem escreve pessoa —, com o gerador determinístico que já existe no repositório (`apps/api/prisma/pessoas-demo.ts`):

- 800.000 usuários em ~2.200 escolas sintéticas (~350 estudantes por escola), turmas de 30, professores lotados;
- liberações criadas para todas as turmas que o cenário usa, porque sem elas a guarda responde 403 e o teste mede o 403;
- **um único *hash* de senha reaproveitado**: 800 mil `bcrypt` de custo 10 custariam ~22 horas de CPU; como o sal viaja dentro do *hash*, a mesma string vale para todos;
- o arquivo de credenciais do gerador fica fora do repositório.

Carregar os 800 mil pelo importador é, de quebra, o primeiro teste: é o volume de uma importação inicial real.

8.3 IA: simulada em escala, real em amostra

A IA custa dinheiro por *token* e tem cota própria. Uma partida da unooGPT custou 71.397 *tokens* (medido em 27/08/2026); 5 mil estudantes jogando ao mesmo tempo seriam ~9 milhões de *tokens* por minuto. Proposta:

- nos testes 3 a 6, o *provider* `mock` com *latência sintética* de 10–30 s — o que interessa ali é quanto a espera prende a API, não o texto (a memória do projeto registra que o *mock* atual lança em `generateJson`, o que precisa ser resolvido antes);
- no teste 7, o Vertex real, com um teto de gasto fixado antes, para medir a cota e a latência sob concorrência. O capítulo de sobrecarga do livro de SRE descreve as ferramentas para quando a cota morder: cota por cliente, *throttling* adaptativo e níveis de criticidade (CUERVO, 2016).

8.4 Banda da escola

O MEC fixou em 2024 a meta de 1 Mbps por aluno no turno de maior matrícula (Conviva Educação, 2024); o NIC.br mediu que só 11% das escolas monitoradas chegam lá e que a média nacional é de 0,26 Mbps por aluno (Comitê Gestor da Internet no Brasil, 2024). Numa escola de 600 estudantes no turno, isso é ~156 Mbps. Se três turmas de 30 abrem juntas uma missão de ~8 MB, são ~720 MB, ou ~37 s com o *link* inteiro dedicado a isso. Os vídeos da `exp04ef1b` somam 52 MB.

Por isso os navegadores do teste híbrido rodam com a rede limitada, e o LCP entra como critério. O Khan Academy atribuiu ter aguentado 2,5 vezes o tráfego em duas semanas, em 2020, a exatamente duas coisas: servidor sem estado e CDN agressiva (KOSARCHYN, 2020) — a mesma aposta da ADR-01 da plataforma.

8.5 Ferramenta

Ferramenta	Roteiro em	Modelo aberto	Navegador real
k6	JavaScript	nativo (`arrival-rate`) (Grafana Labs, 2026e)	sim, com Web Vitals (Grafana Labs, 2026b)
Gatling	Scala, Java, Kotlin	`injectOpen` explícito (Gatling Corp, 2026)	não
Locust	Python	por taxa de usuários (Locust contributors, 2026)	não
JMeter	GUI, Java	por plano	não; trabalha em nível de protocolo e não executa JavaScript (Apache Software Foundation, 2026)

> **DECISÃO DE PROJETO**
>
> **k6.** O roteiro é JavaScript, a linguagem do projeto inteiro; o modelo aberto é nativo; o mesmo roteiro roda os navegadores; e a PU-DOC-001 já o nomeia no `R-01`. A execução distribuída pode ser feita com várias máquinas rodando k6 por fatia do cenário, ou com um serviço gerenciado. O tutorial do Google de teste distribuído com Locust em GKE foi arquivado em agosto de 2026 e não deve ser seguido (Google Cloud Platform, 2026).

Escala do gerador. No modelo aberto, o número de usuários virtuais que o k6 precisa é a taxa de chegada vezes a duração da iteração (Little de novo). Modelar a sessão inteira de 45 minutos como uma iteração exigiria ~50.000 VUs no cenário B; a 1–5 MB por VU, isso é de 50 a 250 GB de memória (Grafana Labs, 2026f). A saída prática é separar os cenários: a jornada completa (J1

e J2) como iteração curta, e o regime (o *polling*, os eventos, o painel) como taxa de requisições por rota, na proporção medida. O Apêndice A mostra o esqueleto.

9

Contas de capacidade para levar ao teste

9.1 Conexões com o banco

Duas regras do Google limitam as conexões: cada instância do Cloud Run abre no máximo 100 conexões com o Cloud SQL (Google Cloud, 2026e), e o `max_connections` padrão do Postgres depende da memória da instância — 100 entre 3,75 e 6 GB, 400 entre 7,5 e 15 GB, 500 entre 15 e 30 GB, 1.000 a partir de 120 GB (Google Cloud, 2026d). A condição a respeitar é:

$$\text{máx. instâncias da API} \times \textit{pool} \text{ por instância} \;\leq\; \texttt{max_connections} - \text{reserva} \tag{9.1}$$

Com o *pool* padrão do Prisma numa instância de 2 vCPU (5 conexões), 100 instâncias pedem 500 conexões — mais que qualquer *tier* até 30 GB. Ou o *pooler* existe de fato (a documentação diz que sim, o repositório não mostra), ou o máximo de instâncias fica preso pelo banco. O Google recomenda *pool* pequeno por instância, com *timeout* e *backoff* exponencial (Google Cloud, 2026g). Este é o primeiro número a confirmar, antes do primeiro teste com mais de uma instância.

9.2 IA pela Lei de Little

Requisições de IA em voo = taxa de perguntas × duração da pergunta. Com 5 mil estudantes na unooGPT (15% do cenário B, arredondado), uma pergunta a cada 5 minutos por estudante e 20 s de espera:

$$N_{\text{IA}} = \frac{5000}{300\,\text{s}} \times 20\,\text{s} \approx 333 \tag{9.2}$$

Na largada, quando 30 estudantes de cada turma perguntam no mesmo minuto, o número sobe para ~1.700. Com concorrência 80 por instância, são de 4 a 21 instâncias fazendo nada além de esperar o Vertex — e cada uma dessas requisições é também um lugar que o estudante do lado, fazendo uma chamada de 50 ms, não consegue ocupar. É o `R-17`; a saída já registrada é *streaming* no *provider*.

9.3 *Login*

A capacidade de *login* por instância é limitada por $1/0{,}1\,\text{s} = 10$ por vCPU enquanto o *hash* rodar no *event loop*. Para os 210/s do cenário B, são pelo menos 21 vCPU só de *hash*, e cada *login* bloqueia por 100 ms todas as outras requisições da mesma instância. Se H1 se confirmar, é a primeira correção a fazer, e ela está dentro da API.

10

Decisões que o teste não toma

O teste mede. Algumas perguntas vêm antes dele e não são técnicas:

1. **A rede de 800 mil é um contrato ou vários?** A ADR-04 define "1 contrato $=$ 1 *stack*", com a escala pensada para ~10.000 estudantes. Uma *stack* para 800 mil muda o *tier* do banco, as cotas e o custo, e concentra o risco: um incidente para a rede inteira. Dividir por regional (várias *stacks* para o mesmo cliente) mantém cada uma perto da escala desenhada, e custa operar várias. **O teste deve rodar sobre a forma escolhida.**
2. **Qual é a adoção esperada (k)?** Uma, duas ou quatro missões por turma por semana mudam a carga em quatro vezes. Isso é do planejamento pedagógico da rede.
3. **Qual é o horário de início das aulas e a divisão entre turnos?** As fontes nacionais não publicam isso em agregado; a rede tem o dado.
4. **Quanto se pode gastar no teste?** Instâncias, banco maior durante o teste e *tokens* de IA do teste 7 precisam de teto aprovado.

> **NOTA**
>
> Ao responder, atualize as premissas P2, P3, P6 e P7 do Capítulo 3: as tabelas de cenário se recalculam com a mesma fórmula.

11

Plano de execução

Com produção prevista para novembro de 2026, sobram cerca de seis semanas.

Semana	Entrega	Sai daqui
1 (até 02/10)	Respostas do Capítulo 10; conferir *pooler*, `connection_limit` e *tier*; pedido de cota; massa sintética carregada pelo importador; *mock* de IA com latência.	Ambiente pronto
2	Roteiros k6 das jornadas J1–J8; teste 1 (*smoke*); teste 2 (capacidade unitária).	Capacidade c por instância
3	Correções das hipóteses confirmadas no teste 2 (provavelmente H1, H2, H3); repetir o teste 2.	c corrigido
4	Teste 3 (carga média) e teste 4 (parede das 8h), com navegadores.	Aprovação no cenário B
5	Teste 5 (estresse e ruptura) e teste 6 (resistência de 5 h).	Ponto de ruptura; vazamentos
6	Teste 7 (IA real); relatório final; atualização da PU-DOC-001 (**RNF-01**, `R-01`, `R-17`, Capítulo 6); limpeza da massa sintética.	`R-01` fechado ou reclassificado

ATENÇÃO

A semana 3 é a que costuma faltar. Todo teste de carga que acha um gargalo precisa de tempo para corrigir e *repetir*; um cronograma sem essa semana só descobre o problema, não o resolve. Os incidentes recorrentes no primeiro dia de inscrição do Enem, atribuídos pela imprensa ao volume de acessos simultâneos, mostram que o risco se repete quando nada muda entre uma edição e a próxima (NERI, 2024; MESSIAS, 2026).

A

Esqueleto do roteiro k6

Esqueleto para orientar a escrita, **não testado**. Os nomes de rota são os do código em 23 de setembro de 2026, e o fluxo de *login* segue o NextAuth com o *provider* de credenciais.

```
import http from "k6/http";
import { check, sleep } from "k6";
import { SharedArray } from "k6/data";

const HUB = __ENV.HUB_URL;
const API = __ENV.API_URL;
const contas = new SharedArray("contas", () =>
  open("./contas-sinteticas.csv").split("\n").slice(1).map((l) => l.split(",")));

export const options = {
  scenarios: {
    // J1 + J2: a parede das 8h (cenario B), modelo aberto.
    parede: {
      executor: "ramping-arrival-rate",
      exec: "entrarNaAula",
      startRate: 0, timeUnit: "1s",
      preAllocatedVUs: 2000, maxVUs: 8000,
      stages: [
        { target: 210, duration: "1m" },  // o degrau
        { target: 210, duration: "1m" },
        { target: 52,  duration: "30s" },
        { target: 52,  duration: "7m30s" },
        { target: 0,   duration: "1m" },
      ],
    },
    // Regime: polling do portao, na taxa por rota (e nao por usuario).
    regime: {
      executor: "constant-arrival-rate",
      exec: "pollingDoPortao",
      rate: 10000, timeUnit: "1s", duration: "30m",
      preAllocatedVUs: 3000, maxVUs: 12000,
      startTime: "10m",
    },
  },
  thresholds: {
    http_req_failed: [{ threshold: "rate<0.005", abortOnFail: true,
                        delayAbortEval: "1m" }],
    "http_req_duration{grupo:api-slo}": ["p(95)<800"],
    "http_req_duration{grupo:login}":   ["p(95)<3000"],
    dropped_iterations: ["count<100"],  // o gerador nao acompanhou
```

```
  },
};

export function entrarNaAula() {
  const [usuario, senha, expId] = contas[Math.floor(Math.random() * contas.length)];
  const csrf = http.get(`${HUB}/api/auth/csrf`, { tags: { grupo: "login" } })
                   .json("csrfToken");
  const r = http.post(`${HUB}/api/auth/callback/credentials`,
    { csrfToken: csrf, username: usuario, password: senha },
    { tags: { grupo: "login" }, redirects: 0 });
  check(r, { "login aceito": (x) => x.status === 302 || x.status === 200 });
  http.get(`${HUB}/estudante`, { tags: { grupo: "login" } });
  http.get(`${HUB}/exp/${expId}`, { tags: { grupo: "abrir" } });
  const token = http.get(`${HUB}/api/exp-token`).json("token");
  http.get(`${API}/rastro-boato/sessao`, {
    headers: { Authorization: `Bearer ${token}` }, tags: { grupo: "api-slo" } });
}

export function pollingDoPortao() {
  // Tokens pre-emitidos para evitar que o regime refaca login.
  // ...
}
```

O detalhe que mais importa está em `dropped_iterations`: no modelo aberto, quando os usuários virtuais pré-alocados acabam, o k6 descarta iterações em vez de desacelerar. Contá-las é o que separa “o sistema aguentou” de “o gerador não conseguiu mandar”.

B

Referências de método sem citação direta no texto

As três obras abaixo são a base clássica de caracterização de carga e planejamento de capacidade, e sustentam o método do Capítulo 3 sem que uma passagem específica tenha sido citada: Jain (1991), Menascé e Almeida (2002) e a norma de qualidade de produto International Organization for Standardization (2023), cuja característica de *eficiência de desempenho* (comportamento no tempo, uso de recursos e capacidade) é a que este teste verifica. O modelo RAIL (Google Chrome Team, 2020) complementa os Web Vitals com os limites de resposta de interação. A classificação do *syllabus* ISTQB e a do guia da Microsoft (MEIER et al., 2007b) coincidem com a do k6 usada no Capítulo 6.

Referências Bibliográficas

Apache Software Foundation. *Apache JMeter*. 2026. <https://jmeter.apache.org/>. Acesso em: 23 set. 2026.

Brasil. *Lei nº 13.709, de 14 de agosto de 2018. Lei Geral de Proteção de Dados Pessoais (LGPD)*. 2018. <https://www.planalto.gov.br/ccivil_03/_ato2015-2018/2018/lei/l13709compilado.htm>. Acesso em: 23 set. 2026.

CARNETT, L. *Technical difficulties plague school districts' online learning platforms as students return to school*. 2020. San Antonio Report. <https://sanantonioreport.org/technical-difficulties-plague-school-districts-online-learning-platforms-as-students-return-to-school/>. Acesso em: 23 set. 2026.

Centro Regional de Estudos para o Desenvolvimento da Sociedade da Informação. *Sete em cada dez alunos do Ensino Médio usam IA generativa em pesquisas escolares, revela TIC Educação*. 2025. Cetic.br. <https://cetic.br/noticia/sete-em-cada-dez-alunos-do-ensino-medio-usam-ia-generativa-em-pesquisas-escolares-revela-tic-educacao/>. Acesso em: 23 set. 2026.

Comitê Gestor da Internet no Brasil. *Levantamento sobre qualidade da Internet em escolas públicas aponta desafios para atingir nova meta de conectividade do MEC*. 2024. <https://www.cgi.br/noticia/releases/levantamento-sobre-qualidade-da-internet-em-escolas-publicas-aponta-desafios-para-atingir-nova-meta-de-conectividade-do-mec/>. Acesso em: 23 set. 2026.

Conviva Educação. *MEC divulga parâmetros de conectividade da Enec*. 2024. <https://convivaeducacao.org.br/fique_atento/4704>. Sobre a Resolução Cenec n. 2/2024. Acesso em: 23 set. 2026.

CUERVO, A. F. Handling overload. In: *Site Reliability Engineering: How Google Runs Production Systems*. Sebastopol: O'Reilly, 2016. cap. 21. Disponível em: <https://sre.google/sre-book/handling-overload/>. Acesso em: 23 set. 2026.

Gatling Corp. *Injection: open and closed workload models*. 2026. Gatling Documentation. <https://docs.gatling.io/concepts/injection/>. Acesso em: 23 set. 2026.

Google. *Web Vitals*. 2026. Web.dev. <https://web.dev/articles/vitals>. Acesso em: 23 set. 2026.

Google Chrome Team. *Measure performance with the RAIL model*. 2020. Web.dev. <https://web.dev/articles/rail>. Acesso em: 23 set. 2026.

Google Cloud. *About instance autoscaling in Cloud Run services*. 2026. Cloud Run Documentation. <https://docs.cloud.google.com/run/docs/about-instance-autoscaling>. Acesso em: 23 set. 2026.

Google Cloud. *Cloud CDN overview*. 2026. Cloud CDN Documentation. <https://docs.cloud.google.com/cdn/docs/overview>. Acesso em: 23 set. 2026.

Google Cloud. *Cloud Security FAQ*. 2026. <https://support.google.com/cloud/answer/6262505?hl=en>. Acesso em: 23 set. 2026.

Google Cloud. *Configure database flags*. 2026. Cloud SQL for PostgreSQL Documentation. <https://docs.cloud.google.com/sql/docs/postgres/flags>. Acesso em: 23 set. 2026.

Google Cloud. *Connect from Cloud Run to Cloud SQL for PostgreSQL*. 2026. Cloud SQL for PostgreSQL Documentation. <https://docs.cloud.google.com/sql/docs/postgres/connect-run>. Acesso em: 23 set. 2026.

Google Cloud. *Load testing best practices*. 2026. Cloud Run Documentation. <https://docs.cloud.google.com/run/docs/about-load-testing>. Acesso em: 23 set. 2026.

Google Cloud. *Manage database connections*. 2026. Cloud SQL for PostgreSQL Documentation. <https://docs.cloud.google.com/sql/docs/postgres/manage-connections>. Acesso em: 23 set. 2026.

Google Cloud Platform. *distributed-load-testing-using-kubernetes*. 2026. GitHub. <https://github.com/GoogleCloudPlatform/distributed-load-testing-using-kubernetes>. Repositório arquivado em ago. 2026. Acesso em: 23 set. 2026.

Grafana Labs. *API load testing*. 2026. Grafana k6 Documentation. <https://grafana.com/docs/k6/latest/testing-guides/api-load-testing/>. Acesso em: 23 set. 2026.

Grafana Labs. *Browser metrics*. 2026. Grafana k6 Documentation. <https://grafana.com/docs/k6/latest/using-k6-browser/metrics/>. Acesso em: 23 set. 2026.

Grafana Labs. *Load test types*. 2026. Grafana k6 Documentation. <https://grafana.com/docs/k6/latest/testing-guides/test-types/>. Acesso em: 23 set. 2026.

Grafana Labs. *Load testing websites*. 2026. Grafana k6 Documentation. <https://grafana.com/docs/k6/latest/testing-guides/load-testing-websites/>. Acesso em: 23 set. 2026.

Grafana Labs. *Open and closed models*. 2026. Grafana k6 Documentation. <https://grafana.com/docs/k6/latest/using-k6/scenarios/concepts/open-vs-closed/>. Acesso em: 23 set. 2026.

Grafana Labs. *Running large tests*. 2026. Grafana k6 Documentation. <https://grafana.com/docs/k6/latest/testing-guides/running-large-tests/>. Acesso em: 23 set. 2026.

Grafana Labs. *Thresholds*. 2026. Grafana k6 Documentation. <https://grafana.com/docs/k6/latest/using-k6/thresholds/>. Acesso em: 23 set. 2026.

GUNTHER, N. J. *Guerrilla Capacity Planning: A Tactical Approach to Planning for Highly Scalable*

Applications and Services. Berlin: Springer, 2007.

Instituto Nacional de Estudos e Pesquisas Educacionais Anísio Teixeira. *Censo Escolar da*

Educação Básica 2024: apresentação coletiva. 2025. Brasília: Inep. <https://download.inep.gov.br/censo_escolar/resultados/2024/apresentacao_coletiva.pdf>. Acesso em: 23 set. 2026.

International Organization for Standardization. *ISO/IEC 25010:2023 — Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — Product quality model*. 2023. <https://www.iso.org/standard/78176.html>. Página de acesso restrito; título conferido por busca. Acesso em: 23 set. 2026.

International Software Testing Qualifications Board. *Certified Tester Foundation Level Specialist Syllabus — Performance Testing*. Version 2018. [S.l.], 2018. Disponível em: <https://istqb.org/wp-content/uploads/2024/11/ISTQB-CT-PT_Syllabus_v1.0_2018.pdf>. Acesso em: 23 set. 2026.

JAIN, R. *The Art of Computer Systems Performance Analysis: Techniques for Experimental Design, Measurement, Simulation, and Modeling*. New York: Wiley-Interscience, 1991.

JONES, C. et al. Service level objectives. In: *Site Reliability Engineering: How Google Runs Production Systems*. Sebastopol: O'Reilly, 2016. cap. 4. Disponível em: <https://sre.google/sre-book/service-level-objectives/>. Acesso em: 23 set. 2026.

KOSARCHYN, M. *How Khan Academy Successfully Handled 2.5x Traffic in a Week*. 2020. Khan Academy Blog. <https://blog.khanacademy.org/how-khan-academy-successfully-handled-2-5x-traffic-in-a-week/>. Acesso em: 23 set. 2026.

LITTLE, J. D. C. A proof for the queuing formula: $L = \lambda W$. *Operations Research*, v. 9, n. 3, p. 383–387, 1961. DOI: 10.1287/opre.9.3.383.

Locust contributors. *What is Locust?* 2026. Locust Documentation. <https://docs.locust.io/en/stable/what-is-locust.html>. Acesso em: 23 set. 2026.

MEIER, J. D. et al. *Chapter 12 — Modeling Application Usage*. 2007. In: Performance Testing Guidance for Web Applications. Microsoft patterns & practices. <https://learn.microsoft.com/en-us/previous-versions/msp-n-p/bb924367(v=pandp.10)>. Acesso em: 23 set. 2026.

MEIER, J. D. et al. *Performance Testing Guidance for Web Applications*. 2007. Microsoft patterns & practices. <https://learn.microsoft.com/en-us/previous-versions/msp-n-p/bb924375(v=pandp.10)>. Acesso em: 23 set. 2026.

MENASCÉ, D. A.; ALMEIDA, V. A. F. *Capacity Planning for Web Services: Metrics, Models, and Methods*. Upper Saddle River: Prentice Hall, 2002.

MESSIAS, L. *Enem 2026: site apresenta instabilidade no 1º dia*. 2026. Quero Bolsa. <https://querobolsa.com.br/revista/enem-2026-site-apresenta-instabilidade-no-1o-dia>. Acesso em: 23 set. 2026.

NERI, Y. *Enem 2024: candidatos reclamam de instabilidade no site do Inep*. 2024. TechTudo. <https://www.techtudo.com.br/dicas-e-tutoriais/2024/05/enem-2024-candidatos-reclamam-de-instabilidade-no-site-do-inep-veja-relatos-edsoftwares.ghtml>. Acesso em: 23 set. 2026.

SCHROEDER, B.; WIERMAN, A.; HARCHOL-BALTER, M. Open versus closed: A cautionary tale.

In: *Proceedings of the 3rd Symposium on Networked Systems Design and Implementation*

(NSDI '06). San Jose: USENIX Association, 2006. p. 239–252. Disponível em: <https://www.usenix.org/legacy/events/nsdi06/tech/full_papers/schroeder/schroeder.pdf>. Acesso em: 23 set. 2026.

TENE, G. *How NOT to Measure Latency*. 2013. Palestra, QCon London 2013. <https://qconlondon.com/london2018/london-2013/qconlondon.com/london-2013/presentation/How%20NOT%20to%20Measure%20Latency.html>. Acesso em: 23 set. 2026.

TENE, G. *wrk2: a constant throughput, correct latency recording variant of wrk*. 2026. GitHub. <https://github.com/giltene/wrk2>. Acesso em: 23 set. 2026.

ULRICH, M. Addressing cascading failures. In: *Site Reliability Engineering: How Google Runs Production Systems*. Sebastopol: O'Reilly, 2016. cap. 22. Disponível em: <https://sre.google/sre-book/addressing-cascading-failures/>. Acesso em: 23 set. 2026.